Rapport de système temps réel

ETN5 - SETR

Élèves :

Nolan BUCHET

Maxence BOULEAU

Date :

Septembre 2026

Table des matières

1 Contexte **2**

2 Etude du code FreeRTOS **3**
2.1 Clignotement de la LED (Partie 3.4) 3
2.2 Réponses à l'étude du code FreeRTOS (Partie 3.5) 3

3 Contrôle de la fréquence de clignotement de la LED **5**

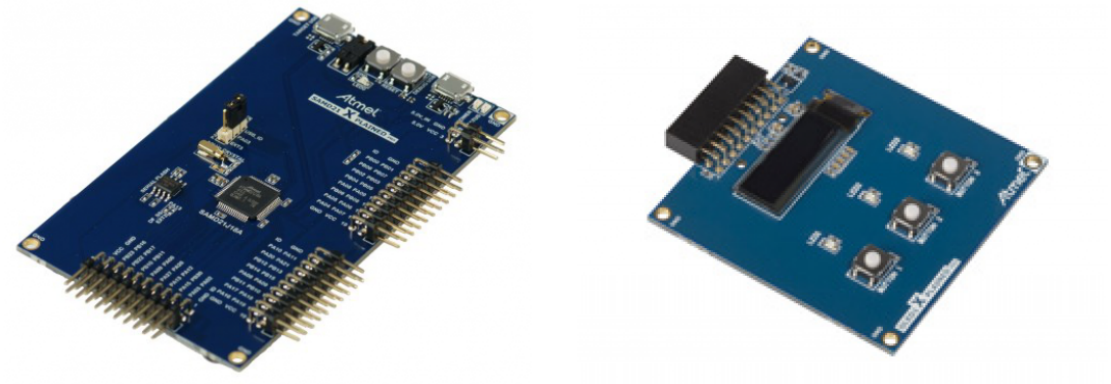
4 Procédure d'intégration d'un RTOS **6**

5 Application à développer **7**
5.1 Contexte 7
5.2 Analyse des besoins en éléments de RTOS pour l'application 7
5.3 Analyse et explication du code de l'application 8
5.4 Preuves de fonctionnement de l'application 8
5.5 Code complet de l'application 8

1 Contexte

Ce projet vise à concevoir un système mesurant le temps de réaction d'un utilisateur à un stimulus visuel, avec une précision d'une milliseconde. L'utilisation du système d'exploitation FreeRTOS garantit le respect de ces contraintes de temps strictes.

Le matériel se compose d'une carte SAMD21XPLAINEDPRO et de son extension OLED1XPLAINEDPRO (écran, LEDs, boutons), programmées sur Microchip Studio. Ce rapport explique l'intégration de FreeRTOS, détaille l'architecture logicielle (tâches, sémaphores) et valide le bon fonctionnement du système.



(a) Carte SAMD21XPLAINEDPRO (b) Extension OLED1XPLAINEDPRO

FIGURE 1 – Matériel utilisé pour le système de mesure du temps de réaction

2 Etude du code FreeRTOS

2.1 Clignotement de la LED (Partie 3.4)

À partir du code fourni en annexe et en suivant les étapes de création du projet sur Microchip Studio, nous avons observé le bon clignotement de la `LED0`.

La tâche `Horloge` définit un délai d'attente de 1000 ms (`HORLOGE_TASK_DELAY 1000`). L'état de la LED changeant toutes les secondes, un cycle complet (allumé puis éteint) dure 2 secondes. Le système fonctionne donc correctement avec une fréquence de clignotement confirmée de 0,5 Hz.

```
void vCodeBlinkLED (void * pvParameters)
{
    /* declaration de variables locales de la tache */
    bool Blink;

    /* initialisation des variables locales */
    Blink = false;

    for (;;)
    {
        /* attente du semaphore d'eveil */
        xSemaphoreTake (Sem, portMAX_DELAY);

        /* Cycle H*/
        if (Blink == false) {
            /* turn LED on. */
            port_pin_set_output_level (LED_0_PIN, LED_0_ACTIVE);
            Blink = true;
        } else {
            /* turn LED off. */
            port_pin_set_output_level (LED_0_PIN, LED_0_ACTIVE);
            Blink = false;
        }
    }
}
```

Le résultat de cette exécution sur la carte SAMD21, confirmant la fréquence de clignotement, est visible via ce lien : **voir la démonstration vidéo sur YouTube**.

2.2 Réponses à l'étude du code FreeRTOS (Partie 3.5)

— **Procédure d'ordonnancement :** Dans le code générique (`tasks.c`), la procédure chargée du changement de contexte et de la sélection de la prochaine tâche à exécuter est `vTaskSwitchContext()` (qui utilise la macro `taskSELECT_HIGHEST_PRIORITY_TASK()`).

— **Lien avec `FreeRTOSConfig.h` :** La politique d'ordonnancement est directement pilotée par les constantes de ce fichier, principalement `configUSE_PREEMPTION` (qui active/désactive le mode préemptif) et `configMAX_PRIORITIES`.

— **Adaptation de la politique :** Adapter fondamentalement la politique (par exemple vers un ordonnancement de type EDF) n'est pas trivial. Il faudrait directement modifier le code source du noyau dans `tasks.c`, en particulier la procédure `vTaskSwitchContext()` et l'organisation des listes de tâches (`pxReadyTasksLists`).

— **Politiques de gestion de la mémoire (heap_1 à heap_5) :**
  — `heap_1.c` : Allocation uniquement, aucune libération possible (déterministe, idéal pour les systèmes statiques).
  — `heap_2.c` : Allocation et libération autorisées, algorithme *best-fit*, mais sans fusion des blocs libres (risque de fragmentation).
  — `heap_3.c` : Encapsule les fonctions standards `malloc()` et `free()` du compilateur pour les rendre *thread-safe*.
  — `heap_4.c` : Allocation et libération avec fusion automatique des blocs adjacents libres (défragmentation).
  — `heap_5.c` : Similaire à `heap_4.c`, mais permet de répartir le tas (heap) sur plusieurs zones mémoires non contiguës.

3 Contrôle de la fréquence de clignotement de la LED

Pour modifier la fréquence de clignotement de la LED dans le projet `OLED1`, plusieurs approches sont possibles à partir du code fourni (`mainLeds-BP_Control.c`) :

— **Modifier la macro de délai (Solution la plus directe) :** La fréquence est régie par la tâche `vCodeClock`, qui s'endort pendant `HORLOGE_TASK_DELAY` ticks (défini à 250) avant de libérer le sémaphore `xSem_H1`. Augmenter ou diminuer la valeur de `HORLOGE_TASK_DELAY` modifiera directement la fréquence de basculement de la LED.

— **Modifier la fréquence du système (Tick Rate) :** La fonction `vTaskDelay()` attend un nombre de "ticks" de l'OS. En modifiant la constante `configTICK_RATE_HZ` dans le fichier `FreeRTOSConfig.h` (initialement fixée à 1000 Hz, soit 1 tick = 1 ms), on modifie la durée temporelle réelle d'un tick, et donc la vitesse d'exécution de toutes les tâches temporisées du système.

— **Ajouter un diviseur logiciel dans la tâche de contrôle :** Dans la tâche `vCodeLEDControl`, au lieu de changer l'état de la LED à chaque réception du sémaphore, on peut introduire un compteur local (variable entière). La LED ne changera d'état que lorsque ce compteur aura atteint un certain seuil, divisant ainsi la fréquence d'horloge.

4 Procédure d'intégration d'un RTOS

Pour intégrer le code source complet de FreeRTOS dans un projet sous Microchip Studio, il faut suivre les étapes suivantes :

1. **Ajout du noyau générique (Sources et Headers) :** Créer un dossier `FreeRTOS` dans le répertoire `src` du projet et y importer les fichiers sources de base (`tasks.c`, `queue.c`, `list.c`, `timers.c`, `event_groups.c`, `stream_buffer.c`, `croutine.c`). Ces fichiers contiennent la logique du RTOS indépendante du matériel. Créer ensuite un sous-dossier `include` pour y placer tous les fichiers d'en-tête (`.h`) associés.

2. **Ajout de la couche matérielle (Portage) :** Créer l'arborescence `FreeRTOS/portable/GCC/ARM_CM` et y importer les fichiers `port.c` et `portmacro.h`. Ces fichiers font le lien direct avec l'architecture du microcontrôleur (Cortex-M0+ de la carte SAMD21) pour gérer les changements de contexte, la pile et les interruptions matérielles.

3. **Ajout du gestionnaire de mémoire :** Créer le dossier `FreeRTOS/portable/MemMang` et y insérer le fichier définissant la politique d'allocation mémoire dynamique du noyau (ici `heap_1.c`).

4. **Configuration du noyau (`FreeRTOSConfig.h`) :** Placer le fichier `FreeRTOSConfig.h` dans le dossier `src/config`. Ce fichier adapte l'OS à l'application en réglant l'horloge CPU (`OSC8M`), la fréquence du tick système (`configTICK_RATE_HZ`), la taille du tas (`configTOTAL_HEAP_SIZE`), le nombre de priorités et la désactivation des fonctions inutilisées (hooks, traces, statistiques). Il fait aussi le lien entre les interruptions du RTOS et celles du processeur (`SysTick_Handler` et `PendSV_Handler`).

5. **Configuration du compilateur (Include Paths) :** Dans les propriétés du projet (*Toolchain* → *ARM/GNU C Compiler* → *Directories*), ajouter les chemins vers les dossiers `FreeRTOS/include` et `FreeRTOS/portable/GCC/ARM_CM0`. Cela permet au compilateur de trouver tous les fichiers `.h` lors de la compilation.

5 Application à développer

5.1 Contexte

Cette partie aborde le développement de l'application finale du projet, qui consiste à implanter un système de mesure du temps de réaction. L'objectif est d'évaluer la vitesse de réaction d'un utilisateur à un stimulus visuel géré par la carte SAMD21 et sa carte d'extension OLED1. Le système doit assurer un comportement fonctionnel et temporel rigoureux en s'appuyant sur les mécanismes de l'exécutif FreeRTOS, tels que les tâches, les sémaphores et les files de messages.

Le comportement séquentiel et temporel de la fonction centrale d'observation est régi par la machine à états présentée dans la figure ci-dessous. Elle gère les différentes phases du système : le repos, l'attente aléatoire d'un stimulus (allumage de la variable *Sortie* sur les LED), la mesure du temps de réaction de l'utilisateur par comptage d'événements, ainsi que le calcul des statistiques (min, max, moyenne) transmises via une file de messages.

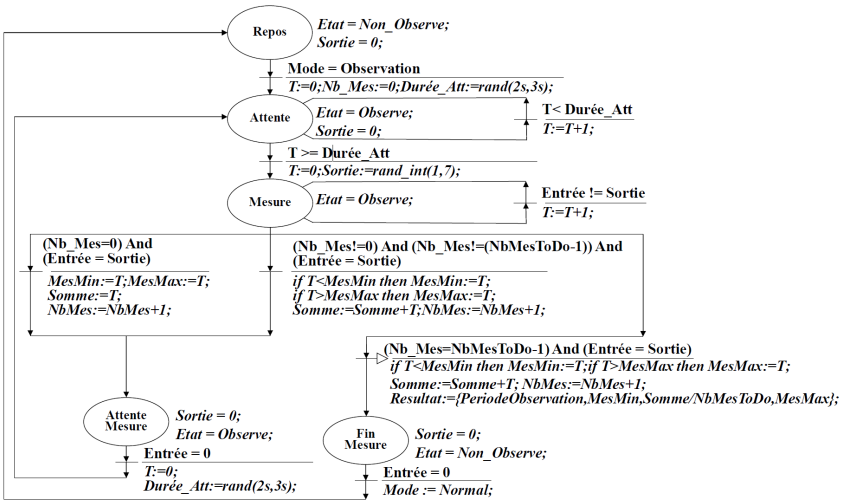


FIGURE 2 – Comportement de la fonction Observation (machine à états)

5.2 Analyse des besoins en éléments de RTOS pour l'application

Pour concevoir cette application sous FreeRTOS, l'analyse fonctionnelle met en évidence plusieurs besoins structurants :

— **Tâches (Tasks) :** L'application est découpée en quatre tâches distinctes hiérarchisées par leurs priorités :

1. `taskClock` (priorité haute) : Tâche périodique (cadencée à 1 ms) servant d'horloge globale de référence.
2. `taskSupervision` : Tâche chargée de scruter l'état du bouton de démarrage pour basculer en mode observation.

3. `taskObservation` : Tâche centrale implémentant l'automate d'état, gérant les stimuli visuels et l'acquisition des boutons.
4. `taskDisplay` : Tâche d'affichage passive restituant les statistiques sur l'écran OLED.

— **Sémaphores binaires :** `SemSupervision` et `SemObservation` synchronisent le réveil des tâches sur le tic d'horloge.

— **Mutex (Exclusion Mutuelle) :** `mutex_mode` et `mutex_oled1` protègent respectivement le mode et les accès concurrents au bus de la carte OLED1.

— **File de messages (Queue) :** `xQueueMeasurementResults` transfère le tableau des résultats statistiques vers l'affichage.

5.3 Analyse et explication du code de l'application

Le programme s'articule autour de l'initialisation matérielle (`system_init`, `oled1_init`, `ConfigureMeasure`), de la création des objets FreeRTOS, puis du lancement de l'ordonnanceur (`vTaskStartScheduler`). Le cœur du système repose sur la machine à états de la tâche d'observation (`REPOS`, `ATTENTE`, `MESURE`, `ATTENTE_MESURE`, `FIN_MESURE`) qui gère l'enchaînement des 5 mesures chronométrées.

5.4 Preuves de fonctionnement de l'application

Le bon comportement temporel de l'application a été validé sur la cible SAMD21 Xplained Pro avec la carte OLED1, garantissant une période d'observation stable voisine de 1 ms.

Le résultat de cette exécution sur la carte SAMD21, confirmant le bon fonctionnement de l'application, est visible via ce lien : **voir la démonstration vidéo sur YouTube**.

5.5 Code complet de l'application

Le code source complet de l'application temps-réel FreeRTOS est présenté ci-dessous :

```
/* Inclusion des bibliothèques */
#include <asf.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include "timers.h"

#include "oled1.h"
```

```
#include "MesTemps.h"
#include "gfx_mono.h"
#include "gfx_mono_text.h"
#include "sysfont.h"

#define OLED1_EXT_HEADER      EXT1
static OLED1_CREATE_INSTANCE(oled1, OLED1_EXT_HEADER);

#define TASK_CLOCK_PRIORITY        (tskIDLE_PRIORITY + 4)
#define TASK_SUPERVISION_PRIORITY  (tskIDLE_PRIORITY + 3)
#define TASK_OBSERVATION_PRIORITY  (tskIDLE_PRIORITY + 2)
#define TASK_DISPLAY_PRIORITY      (tskIDLE_PRIORITY + 1)

#define HORLOGE_TASK_DELAY         1
#define NBMES_TO_DO                5

static SemaphoreHandle_t SemSupervision = NULL;
static SemaphoreHandle_t SemObservation = NULL;
static SemaphoreHandle_t mutex_mode     = NULL;
static SemaphoreHandle_t mutex_oled1    = NULL;
static QueueHandle_t xQueueMeasurementResults = NULL;

void vSetupHardware(void);
void taskClock(void *pvParameters);
void taskSupervision(void *pvParameters);
void taskObservation(void *pvParameters);
void taskDisplay(void *pvParameters);

typedef enum {
    REPOS,
    ATTENTE,
    MESURE,
    ATTENTE_MESURE,
    FIN_MESURE
} StateForObservation;

typedef enum {
    Normal,
    Observation
} Mode;

static Mode mode = Normal;
static uint16_t measurementResults[4];

static uint16_t T = 0;
static uint16_t Nb_Mes = 0;
```

```
static uint16_t Duree_Att = 0;
static uint16_t MesMin = 0;
static uint16_t MesMax = 0;
static uint32_t Somme = 0;
static uint8_t  Sortie_val = 0;

uint8_t get_entree_buttons(void)
{
    uint8_t val = 0;
    if (xSemaphoreTake(mutex_oled1, portMAX_DELAY) == pdTRUE) {
        if (oled1_get_button_state(&oled1, OLED1_BUTTON1_ID)) val |= 0x01;
        if (oled1_get_button_state(&oled1, OLED1_BUTTON2_ID)) val |= 0x02;
        if (oled1_get_button_state(&oled1, OLED1_BUTTON3_ID)) val |= 0x04;
        xSemaphoreGive(mutex_oled1);
    }
    return val;
}

void set_sortie_leds(uint8_t val)
{
    if (xSemaphoreTake(mutex_oled1, portMAX_DELAY) == pdTRUE) {
        oled1_set_led_state(&oled1, OLED1_LED1_ID, (val & 0x01) ? true : false);
        oled1_set_led_state(&oled1, OLED1_LED2_ID, (val & 0x02) ? true : false);
        oled1_set_led_state(&oled1, OLED1_LED3_ID, (val & 0x04) ? true : false);
        xSemaphoreGive(mutex_oled1);
    }
}

void set_etat_led0(bool observe)
{
    if (observe) {
        port_pin_set_output_level(LED_0_PIN, LED_0_ACTIVE);
    } else {
        port_pin_set_output_level(LED_0_PIN, !LED_0_ACTIVE);
    }
}

void vSetupHardware(void)
{
    system_init();
    gfx_mono_init();
    oled1_init(&oled1);
    ConfigureMeasure();
}

int main(void)
```

```
{
    vSetupHardware();

    gfx_mono_draw_filled_rect(0, 0, GFX_MONO_LCD_WIDTH, GFX_MONO_LCD_HEIGHT, GFX_PIXEL_C
    gfx_mono_draw_string("READY - Press SW0", 0, 0, &sysfont);

    srand(1234);

    SemSupervision = xSemaphoreCreateBinary();
    SemObservation = xSemaphoreCreateBinary();
    mutex_mode     = xSemaphoreCreateMutex();
    mutex_oled1    = xSemaphoreCreateMutex();
    xQueueMeasurementResults = xQueueCreate(1, sizeof(measurementResults));

    xTaskCreate(taskClock,       "Clock",       configMINIMAL_STACK_SIZE,     NULL, TASK
    xTaskCreate(taskSupervision, "Supervision", configMINIMAL_STACK_SIZE,     NULL, TASK
    xTaskCreate(taskObservation, "Observation", configMINIMAL_STACK_SIZE * 2, NULL, TASK
    xTaskCreate(taskDisplay,     "Display",     configMINIMAL_STACK_SIZE * 3, NULL, TASK

    StartMeasure();
    vTaskStartScheduler();

    for (;;);
}

void taskClock(void *pvParameters)
{
    (void) pvParameters;
    for (;;)
    {
        vTaskDelay(pdMS_TO_TICKS(HORLOGE_TASK_DELAY));
        xSemaphoreGive(SemSupervision);
        xSemaphoreGive(SemObservation);
    }
}

void taskSupervision(void *pvParameters)
{
    (void) pvParameters;
    bool Start;
    bool StartPrec = false;

    for (;;)
    {
        xSemaphoreTake(SemSupervision, portMAX_DELAY);
        Start = (port_pin_get_input_level(BUTTON_0_PIN) == BUTTON_0_ACTIVE);
```

```
        if ((StartPrec == false) && (Start == true))
        {
            if (xSemaphoreTake(mutex_mode, portMAX_DELAY) == pdTRUE)
            {
                if (mode == Normal) mode = Observation;
                xSemaphoreGive(mutex_mode);
            }
        }
        StartPrec = Start;
    }
}

void taskObservation(void *pvParameters)
{
    (void) pvParameters;
    StateForObservation PresState = REPOS;
    StateForObservation NextState = REPOS;
    uint16_t periodeObservation_us = 0;

    for (;;)
    {
        xSemaphoreTake(SemObservation, portMAX_DELAY);
        periodeObservation_us = EndMeasure();
        StartMeasure();

        PresState = NextState;

        Mode current_mode = Normal;
        if (xSemaphoreTake(mutex_mode, portMAX_DELAY) == pdTRUE) {
            current_mode = mode;
            xSemaphoreGive(mutex_mode);
        }

        switch (PresState)
        {
            case REPOS:
                set_etat_led0(false);
                set_sortie_leds(0);
                if (current_mode == Observation)
                {
                    T = 0;
                    Nb_Mes = 0;
                    Duree_Att = 2000 + (rand() % 1001);
                    NextState = ATTENTE;
                }
```

```
        break;
    case ATTENTE:
        set_etat_led0(true);
        set_sortie_leds(0);
        if (T < Duree_Att)
        {
            T = T + 1;
            NextState = ATTENTE;
        }
        else
        {
            T = 0;
            Sortie_val = 1 + (rand() % 7);
            set_sortie_leds(Sortie_val);
            NextState = MESURE;
        }
        break;

    case MESURE:
    {
        set_etat_led0(true);
        uint8_t entree = get_entree_buttons();

        if (entree != Sortie_val)
        {
            T = T + 1;
            NextState = MESURE;
        }
        else
        {
            if (Nb_Mes == 0)
            {
                MesMin = T; MesMax = T; Somme = T;
                Nb_Mes++; set_sortie_leds(0);
                NextState = ATTENTE_MESURE;
            }
            else if (Nb_Mes < (NBMES_TO_DO - 1))
            {
                if (T < MesMin) MesMin = T;
                if (T > MesMax) MesMax = T;
                Somme += T; Nb_Mes++; set_sortie_leds(0);
                NextState = ATTENTE_MESURE;
            }
            else
            {
                if (T < MesMin) MesMin = T;
```

```
                        if (T > MesMax) MesMax = T;
                        Somme += T; Nb_Mes++;

                        measurementResults[0] = periodeObservation_us;
                        measurementResults[1] = MesMin;
                        measurementResults[2] = (uint16_t)(Somme / NBMES_TO_DO);
                        measurementResults[3] = MesMax;

                        xQueueSend(xQueueMeasurementResults, &measurementResults, 0);
                        set_sortie_leds(0);
                        set_etat_led0(false);
                        NextState = FIN_MESURE;
                    }
                }
                break;
            }

            case ATTENTE_MESURE:
                set_sortie_leds(0);
                set_etat_led0(true);
                if (get_entree_buttons() == 0)
                {
                    T = 0;
                    Duree_Att = 2000 + (rand() % 1001);
                    NextState = ATTENTE;
                }
                break;

            case FIN_MESURE:
                set_sortie_leds(0);
                set_etat_led0(false);
                if (get_entree_buttons() == 0)
                {
                    if (xSemaphoreTake(mutex_mode, portMAX_DELAY) == pdTRUE) {
                        mode = Normal;
                        xSemaphoreGive(mutex_mode);
                    }
                    NextState = REPOS;
                }
                break;
        }
    }
}

void taskDisplay(void *pvParameters)
{
```

```
    (void) pvParameters;
    char buf[24];
    uint16_t receivedResults[4];

    for (;;)
    {
        if (xQueueReceive(xQueueMeasurementResults, &receivedResults, portMAX_DELAY) == 
        {
            if (xSemaphoreTake(mutex_oled1, portMAX_DELAY) == pdTRUE)
            {
                gfx_mono_draw_filled_rect(0, 0, GFX_MONO_LCD_WIDTH, GFX_MONO_LCD_HEIGHT,

                snprintf(buf, sizeof(buf), "Per: %u us", receivedResults[0]);
                gfx_mono_draw_string(buf, 0, 0, &sysfont);

                snprintf(buf, sizeof(buf), "Min: %u ms", receivedResults[1]);
                gfx_mono_draw_string(buf, 0, 8, &sysfont);

                snprintf(buf, sizeof(buf), "Moy: %u ms", receivedResults[2]);
                gfx_mono_draw_string(buf, 0, 16, &sysfont);

                snprintf(buf, sizeof(buf), "Max: %u ms", receivedResults[3]);
                gfx_mono_draw_string(buf, 0, 24, &sysfont);

                xSemaphoreGive(mutex_oled1);
            }
        }
    }
}
```